

Installation chez un nouveau client

Traceability 2.0 | fiche de procedure | mise a jour du 06/09/2026

PHASE 0 | La cle USB d'identifiants (une seule fois)

- ☐ Sur un Pi deja installe, creer le modele :

```
python3 ~/traceability-app/tools/finaliser.py --exemple
```
- ☐ Renseigner le jeton Telegram, le chat_id et, si besoin, les cles Tuya. Ces identifiants sont les MEMES sur toutes les machines.
- ☐ Copier le fichier **identifiants.txt** sur une cle USB. Elle servira a toutes les installations suivantes.

A PROTEGER Cette cle donne le controle du bot Telegram et l'accès au compte capteurs Tuya. La garder a l'atelier : ne jamais l'oublier chez un client.

PHASE 1 | Preparer la carte SD (~10 min)

- ☐ Carte de **16 Go minimum** (une installation complete occupe 7,5 Go). Acheter les cartes par lot identique.
- ☐ Raspberry Pi Imager : **Raspberry Pi OS avec bureau** (version proposee par default).
- ☐ **Modifier les reglages** avant d'ecrire :
 - ☐ • **Nom d'hote** = nom du client, en minuscules avec tirets (exemple : boucherie-durand)
 - ☐ • **WiFi** du magasin (reseau 2,4 GHz uniquement)
 - ☐ • **Nom d'utilisateur** : libre, le script s'adapte
- ☐ Ecrire l'image sur la carte.

ATTENTION Le nom d'hote est l'identifiant du Pi pour le verrouillage et les mises a jour a distance. Il doit etre unique et facile a reconnaitre.

PHASE 2 | Installer l'application (~45 min sans surveillance)

- ☐ Carte dans le Pi, ecran, alimentation. Demarrer et ouvrir un terminal.
- ☐ Lancer l'installation :

```
git clone https://github.com/halil6938/traceability-2.0.git
cd traceability-2.0
bash install.sh
sudo reboot
```
- ☐ Le script installe les paquets, les reglages camera (dtoverlay=ov5647,vcm) et le demarrage automatique. Le redemarrage est indispensable : sans lui, l'autofocus ne fonctionne pas.

GAIN DE TEMPS Plusieurs Pi peuvent etre installes **EN MEME TEMPS** : lancer la commande sur chacun, puis les laisser travailler en parallele. C'est le principal gain face au clonage de carte, qui monopolise le PC carte apres carte.

PHASE 3 | Finaliser la machine (~5 min)

- ☐ Brancher la cle USB d'identifiants, puis :

```
python3 ~/traceability-app/tools/finaliser.py
```

- ☐ Jeton Telegram et cles Tuya sont appliques en une fois. Verifier avec **--etat** ; un message Telegram confirme l'installation.
- ☐ **Orientation de l'ecran** : Menu > Preferences > Screen Configuration, clic droit sur l'ecran > Orientation > **Inverted** (180 degrees), puis Appliquer.
- ☐ Verifier le nom de la machine :

```
hostname
```
- ☐ S'il est incorrect :

```
bash ~/traceability-app/tools/set_client.sh boucherie-durand
sudo reboot
```

PHASE 4 | Configuration dans l'application (~15 min)

a) Les appareils (frigos et congelateurs)

- ☐ L'assistant du premier demarrage demande d'en creer un : nom, temperature MIN, temperature MAX. Ces seuils declenchent les alertes.
- ☐ Pour les suivants : **Parametres > + Ajouter**.

b) Les capteurs de temperature (Parametres > Capteurs temp.)

- ☐ **Ajouter BLE** : capteurs allumes et proches. Ils s'affichent avec leur temperature actuelle, ce qui permet d'identifier lequel est dans quel frigo. Le bouton **Adresse** permet une saisie manuelle.
- ☐ **Ajouter WiFi** (capteurs Tuya) : le capteur doit d'abord etre appaire dans l'application Smart Life du fournisseur, sinon il n'apparait pas dans la liste.
- ☐ **Assigner** chaque capteur a son appareil, puis **Tester** pour verifier que les temperatures remontent.

c) La reception (Reception > Fournisseurs)

- ☐ Saisir les fournisseurs du client.
- ☐ **Pistolet BLE** puis **Detecter** : appuyer sur la gachette pendant la recherche. La MAC peut aussi etre saisie a la main.

d) La camera (Parametres > Test camera)

- ☐ Placer un ticket imprime bien eclaire a sa distance definitive, puis **Calibrer** (environ 10 s, sans rien bouger).
- ☐ Verifier ensuite un scan reel, et la lisibilite de la photo.

ATTENTION La mise au point depend du montage : la calibration est a refaire sur CHAQUE installation.

PHASE 5 | Enregistrer le client pour le controle a distance

- ☐ Sur GitHub, dossier **devices/**, creer le fichier **<nom-du-client>.json** en copiant _modele.json :

```
{ "locked": false, "message": "", "config": {}, "update": "auto" }
```
- ☐ Pour bloquer ce client : passer **locked** a true et ecrire un message. Effet en 20 secondes environ ; le blocage resiste au redemarrage et a une coupure internet.
- ☐ **update** : auto (mises a jour la nuit, recommande), now (immédiat) ou off (fige la version).

MEMO | Depannage et verifications

Version installee et nom du Pi : en bas de l'ecran **Parametres**.

```
python3 ~/traceability-app/tools/finaliser.py --etat # identifiants
sudo systemctl restart traceability # redemarrer l'application
journalctl -u traceability -n 40 # journal de l'application
cat ~/traceability/logs/update.log # mises a jour
cat ~/traceability/logs/ble_thermo.log # pistolet infrarouge
python3 ~/traceability-app/tools/focus_probe.py # test du focus
```

Le code se met a jour tout seul depuis GitHub : rien a transferer sur les Pi apres une correction.

Signe de vie Telegram : chaque Pi annonce son installation, ses mises a jour, et envoie un message quotidien.
Un Pi silencieux est hors ligne.

L'ecran se met en veille apres 10 minutes sans contact. Le premier contact rallume seulement : il ne declenche aucun bouton. Delai reglable a distance (SCREEN_OFF_S).